Personalized Networking Assistant: AI-Powered Conversation Starter Generator

Project Description:

The Personalized Networking Assistant is an AI-powered web application that helps users generate smart, tailored conversation starters for professional and social networking events. The platform includes an Event Analyzer module, which extracts key themes from an event description using DistilBERT zero-shot classification, a Conversation Generator module, which produces two to three context-aware conversation prompts using GPT-2, and a Fact Checker module, which retrieves a summarized, reliable reference on any topic using the Wikipedia API.

Built with FastAPI for the backend and Streamlit for the frontend, the application follows a modular, service-oriented architecture in which each core capability is implemented as an independent, testable component. Conversation history and user feedback are persisted locally in JSON files, allowing users to revisit past suggestions and mark which ones were useful. All backend services are unit-tested using pytest, and API endpoints are verified through FastAPI's Swagger UI, ensuring a reliable and maintainable solution for real-world networking scenarios.

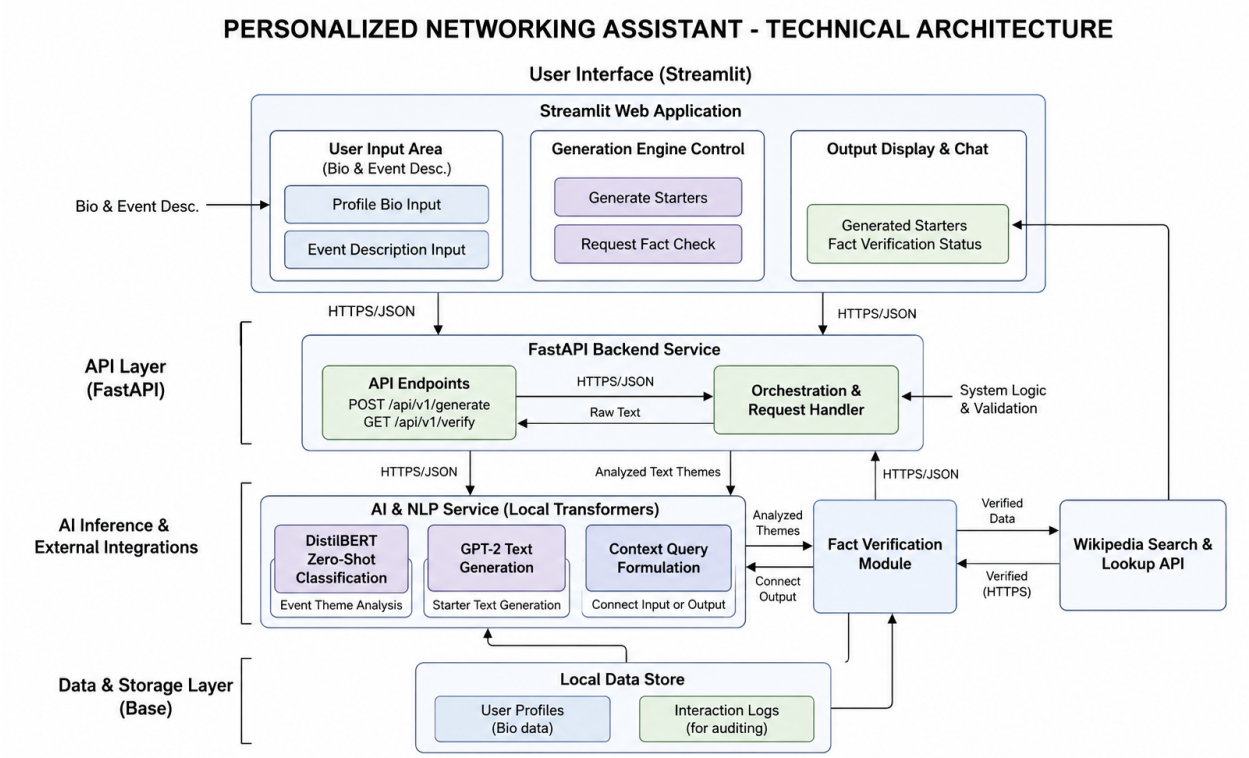
Scenarios

Scenario 1: A user enters an event description such as “AI for Sustainable Cities” and specifies interests like “climate change” and “urban planning.” The assistant extracts relevant themes (e.g., AI, sustainability) and generates two to three conversation starters aligned with the user's goals.

Scenario 2: A user preparing to attend a conference searches for a quick fact check on “blockchain in healthcare.” The application provides a summarized, reliable reference using the Wikipedia API within seconds.

Scenario 3: A user accesses the “History” feature to review previously generated conversations and see which ones were marked useful with a thumbs-up. This feedback loop encourages continuous improvement in the user's networking strategy over time.

Technical Architecture:



Description: The Personalized Networking Assistant follows a layered architecture. The Streamlit web application forms the user interface layer, capturing the user's bio and event description and displaying generated starters, fact-verification status, and conversation history. The FastAPI backend service exposes REST endpoints that orchestrate requests between the frontend and the underlying AI and data layers. The AI and NLP service layer runs DistilBERT for zero-shot theme classification and GPT-2 for conversation-starter text generation, and forwards analyzed themes to the Fact Verification module, which queries the Wikipedia Search and Lookup API to return verified reference data. The local data store persists user profile and interaction logs to support history review and auditing.
(Note: The current solution uses local JSON files for persistence rather than a relational database; if a database is introduced in a future iteration, an ER diagram should be added here along with a description of the schema.)

Pre-requisites:

- **FastAPI Framework Knowledge:** FastAPI Documentation
- **Hugging Face Transformers Familiarity:** Transformers Documentation
- **Streamlit Basics:** Streamlit Documentation
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Wikipedia API Wrapper Usage:** Wikipedia-API Python Package Documentation
- **Testing with pytest and httpx:** pytest Documentation

Project Workflow:

Milestone 1: Environment Setup & Dependency Configuration

Milestone 2: Model Selection & Architecture

Milestone 3: Core Backend Development

Milestone 4: Data Persistence & Logging

Milestone 5: Streamlit Frontend UI Development

Milestone 6: Testing & Deployment

Milestone 1: Environment Setup & Dependency Configuration

This milestone focuses on preparing a clean, reproducible development environment for the Personalized Networking Assistant, ensuring that all required libraries for NLP, the backend API, and testing are correctly installed before development begins.

Activity 1.1: Set Up the Python Environment

- Install Python 3.11+ and confirm pip is available for dependency management.
- Create and activate a virtual environment to isolate project dependencies.

```
python -m venv venv
venv\Scripts\activate      (Windows)
source venv/bin/activate    (macOS/Linux)
```

Activity 1.2: Install Required Libraries

- Install FastAPI and Uvicorn for the backend service.
- Install Streamlit for the frontend application.
- Install Hugging Face Transformers for DistilBERT and GPT-2 models.
- Install the Wikipedia API Python wrapper for fact-checking.
- Install pytest and httpx for automated testing.

```
pip install fastapi uvicorn streamlit
pip install transformers torch
pip install wikipedia-api
pip install pytest httpx
```

Activity 1.3: Initialize Version Control

- Initialize a Git repository and create an initial commit with the base project structure.

```
git init
git add .
git commit -m "Initial project setup"
```

Milestone 2: Model Selection & Architecture

This milestone focuses on evaluating and integrating the Hugging Face models that power the assistant's core NLP capabilities: theme extraction and conversation generation.

Activity 2.1: Theme Extraction with DistilBERT

• Use DistilBERT for fast zero-shot classification to extract themes from event descriptions without needing a custom-labeled training set.

```
from transformers import pipeline

theme_classifier = pipeline(
    "zero-shot-classification",
    model="distilbert-base-uncased"
)

def extract_themes(event_description: str, candidate_labels: list) -> list:
    result = theme_classifier(event_description, candidate_labels)
    return result["labels"][:3]
```

Activity 2.2: Conversation Generation with GPT-2

• Integrate GPT-2 Small using the text-generation pipeline to produce human-like, context-aware conversation prompts based on the extracted themes and user interests.

```
from transformers import pipeline

conversation_generator = pipeline("text-generation", model="gpt2")

def generate_starters(themes: list, interests: list, num_starters: int = 3) -> list:
    prompt = f"Event themes: {', '.join(themes)}. Interests: {', '.join(interests)}.\n"
    outputs = conversation_generator(
        prompt, max_length=60, num_return_sequences=num_starters
    )
    return [o["generated_text"].strip() for o in outputs]
```

Milestone 3: Core Backend Development

This milestone builds the modular FastAPI services and routes that expose the assistant's core functionality to the frontend.

Activity 3.1: Build the Event Analyzer Module

• Implement event_analyzer.py to wrap the DistilBERT theme extraction logic as a reusable service.

Activity 3.2: Build the Conversation Generator Module

• Implement topic_generator.py to wrap the GPT-2 generation logic and format the output for the API response.

Activity 3.3: Build the Fact-Checking Module

• Implement fact_checker.py using the Wikipedia API to retrieve a concise, reliable summary for a searched topic.

```
import wikipediaapi

wiki = wikipediaapi.Wikipedia(user_agent="NetworkingAssistant/1.0", language="en")

def verify_fact(topic: str) -> str:
    page = wiki.page(topic)
    if not page.exists():
        return "No reliable summary found for this topic."
    return page.summary[:500]
```

Activity 3.4: Define API Routes

• Define the /analyze-event, /generate-conversation, and /fact-check routes in the FastAPI application, each delegating to its corresponding service module.

```
from fastapi import FastAPI
from pydantic import BaseModel
from app.services.event_analyzer import extract_themes
from app.services.topic_generator import generate_starters
from app.services.fact_checker import verify_fact

app = FastAPI(title="Personalized Networking Assistant")

class EventRequest(BaseModel):
    event_description: str
    interests: list[str]

@app.post("/analyze-event")
def analyze_event(request: EventRequest):
    themes = extract_themes(request.event_description, request.interests)
    return {"themes": themes}

@app.post("/generate-conversation")
def generate_conversation(request: EventRequest):
    themes = extract_themes(request.event_description, request.interests)
    starters = generate_starters(themes, request.interests)
    return {"themes": themes, "starters": starters}

@app.get("/fact-check")
def fact_check(topic: str):
    return {"topic": topic, "summary": verify_fact(topic)}
```

Milestone 4: Data Persistence & Logging

This milestone implements lightweight local data storage so users can review previously generated conversations and the feedback they provided.

Activity 4.1: Conversation History Logging

- Implement history_logger.py to append each generated conversation to history.json.

```
import json
from datetime import datetime

HISTORY_FILE = "data/history.json"

def log_conversation(event_description: str, starters: list):
    entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "event_description": event_description,
        "starters": starters,
    }
    try:
        with open(HISTORY_FILE, "r") as f:
            history = json.load(f)
    except FileNotFoundError:
        history = []
    history.append(entry)
    with open(HISTORY_FILE, "w") as f:
        json.dump(history, f, indent=2)
```

Activity 4.2: Feedback Logging

- Implement feedback_logger.py to capture thumbs-up / thumbs-down actions in feedback.json for future personalization improvements.

```
def log_feedback(starter: str, is_useful: bool):
    entry = {"starter": starter, "useful": is_useful}
    try:
        with open("data/feedback.json", "r") as f:
            feedback = json.load(f)
    except FileNotFoundError:
        feedback = []
    feedback.append(entry)
    with open("data/feedback.json", "w") as f:
        json.dump(feedback, f, indent=2)
```

Milestone 5: Streamlit Frontend UI Development

This milestone designs an interactive Streamlit interface that lets users enter event details, trigger generation, and review their conversation history and feedback in real time.

Activity 5.1: Build the Input Section

- Create input fields for the event description and user interests, along with a button to trigger conversation generation.

```
import streamlit as st
import requests

st.title("Personalized Networking Assistant")

event_description = st.text_area("Event Description")
interests = st.text_input("Your Interests (comma-separated)")

if st.button("Generate Conversation Starters"):
    payload = {
        "event_description": event_description,
        "interests": [i.strip() for i in interests.split(",")],
    }
    response = requests.post(
        "http://localhost:8000/generate-conversation", json=payload
    )
    st.session_state["result"] = response.json()
```

Activity 5.2: Display Results and History

- Render the generated starters with thumbs-up / thumbs-down feedback buttons, and display recent conversation history retrieved from history.json.

```
if "result" in st.session_state:
    for starter in st.session_state["result"]["starters"]:
        st.write(starter)
        col1, col2 = st.columns(2)
        col1.button("■■ Useful", key=f"up_{starter}")
        col2.button("■■ Not Useful", key=f"down_{starter}")
```

Milestone 6: Testing & Deployment

This milestone validates the solution end-to-end and prepares it for local deployment and demonstration.

Activity 6.1: Unit Testing with pytest

- Write unit tests for the event analyzer, conversation generator, and fact-checking services.

```
def test_extract_themes():
    themes = extract_themes(
        "AI for Sustainable Cities", ["AI", "sustainability", "finance"]
    )
    assert "AI" in themes or "sustainability" in themes

def test_generate_starters():
    starters = generate_starters(["AI", "sustainability"], ["climate change"])
    assert len(starters) == 3
```

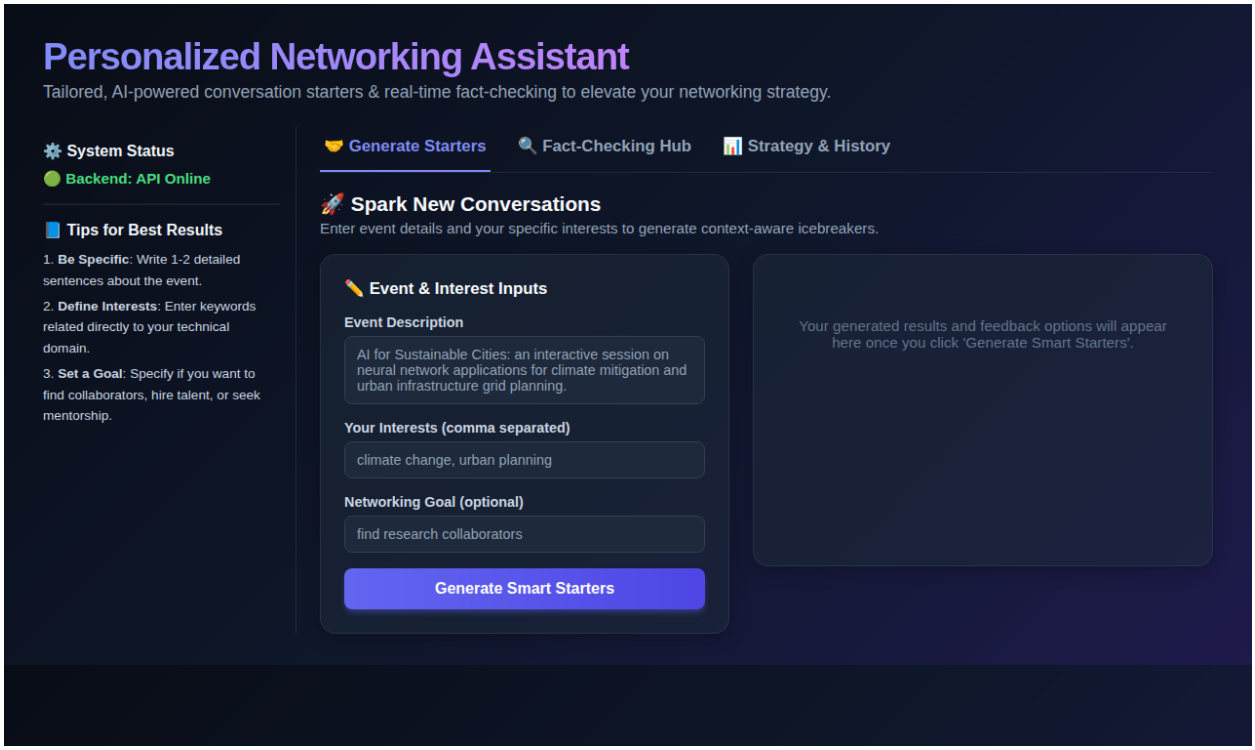
Activity 6.2: API Testing and Local Deployment

- Test all API endpoints using FastAPI's Swagger UI at http://localhost:8000/docs.
- Run integration tests against the live API using httpx.
- Start the FastAPI backend and Streamlit frontend together for local deployment.

```
uvicorn app.main:app --reload
streamlit run frontend/app.py
```

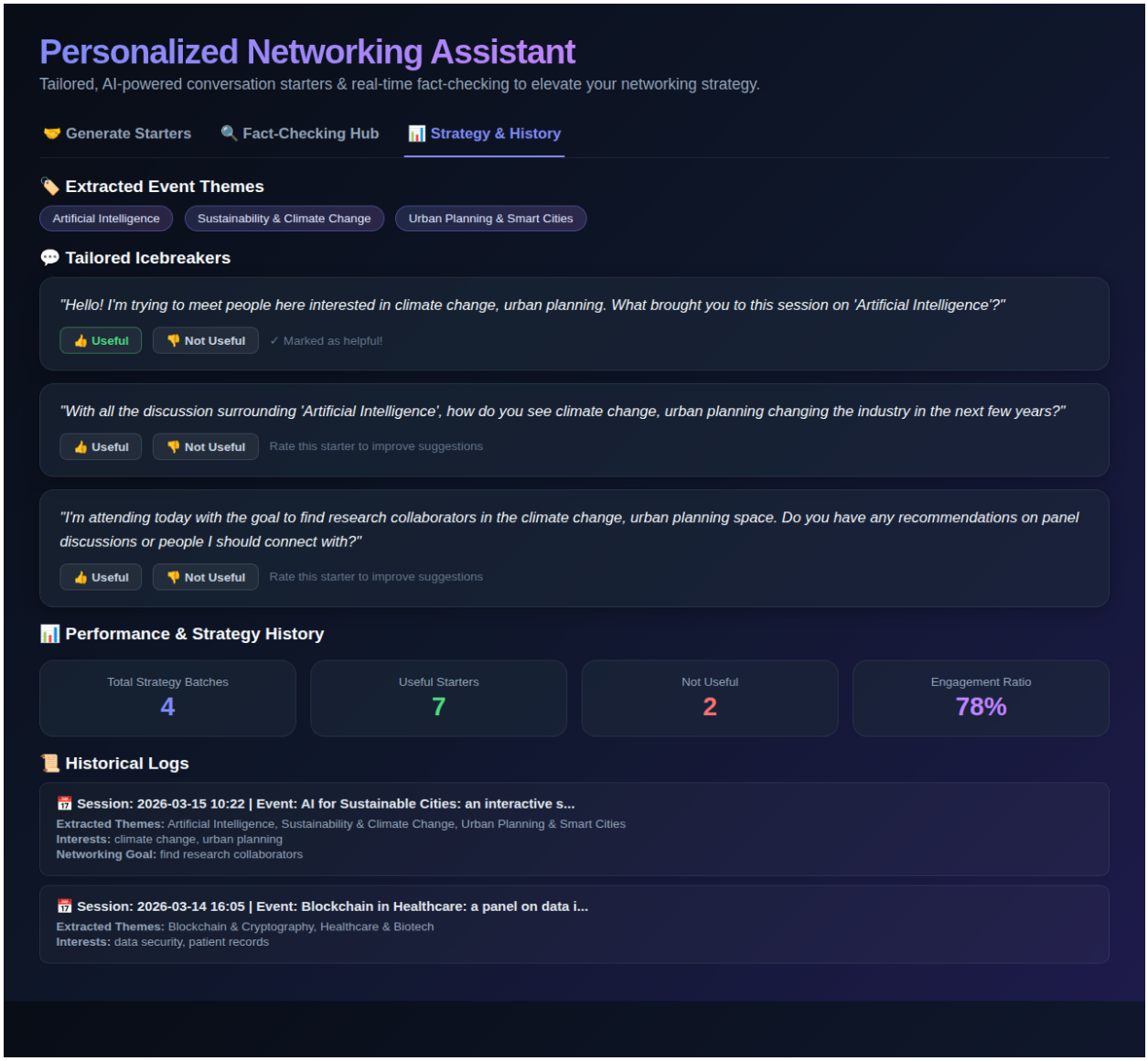
Exploring the Website's Web Pages:

Home / Input Page:



Description: The main Streamlit page presents an input section where the user enters the event description and their interests, followed by a button to generate conversation starters. A separate input field allows users to search for a quick fact check on any topic.

Results & History Page:



Description: Once generated, conversation starters are displayed with thumbs-up and thumbs-down feedback controls next to each suggestion. A History section below lists previously generated conversations along with their recorded feedback, allowing the user to review and refine their networking strategy over time.

Conclusion:

The Personalized Networking Assistant is an AI-powered platform built with FastAPI and Streamlit that helps users generate smart, tailored conversation starters for networking events. It combines DistilBERT for theme extraction, GPT-2 for conversation generation, and the Wikipedia API for fact verification within a modular, well-tested backend architecture. The project, which spanned environment setup, model integration, backend and frontend development, and local testing and deployment, demonstrates how targeted NLP models can be combined into a practical tool that improves users' networking outcomes. Looking ahead, the assistant offers room for growth, including user authentication, multi-language conversation generation, and cloud deployment to make the tool accessible to a wider audience.